Capstone Project: Make the Computer Draw a Box

Nand to Tetris - I (ETCCNT301)

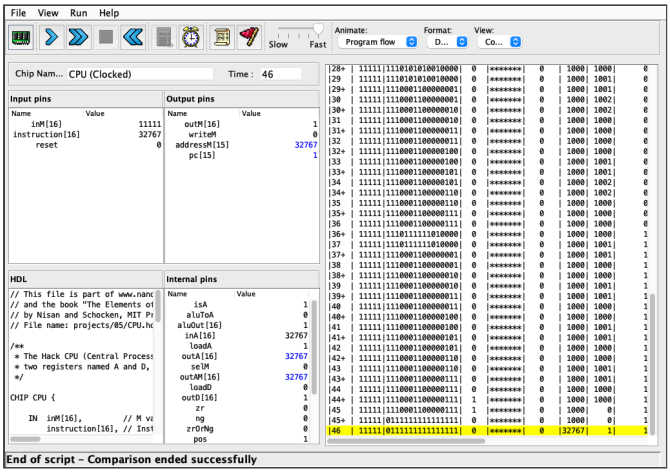
School of Engineering & Technology, K.R. Mangalam University

Name: Prachi Manwal	**Roll No.:** 2501730365	**Personal Number (K):** 65
Box Position & Size (for K = 65): Starts at Row 65 (address 18464), 20 rows tall, 16 × ((65 mod 4) + 1) = 32 pixels wide (2 words from left edge)		
Assembler Extra Feature (K mod 3 = 65 mod 3 = 2): Print a warning if the same label is written twice		

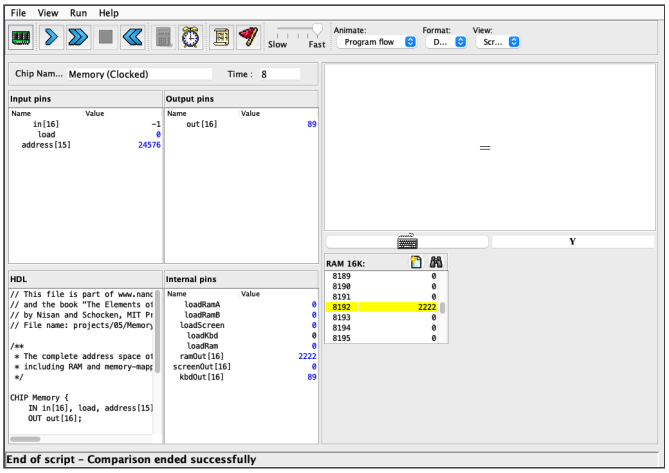
Part A - Build the Computer

In Part A, I connected the chips from earlier labs to build `CPU.hdl`, `Memory.hdl`, and `Computer.hdl`.

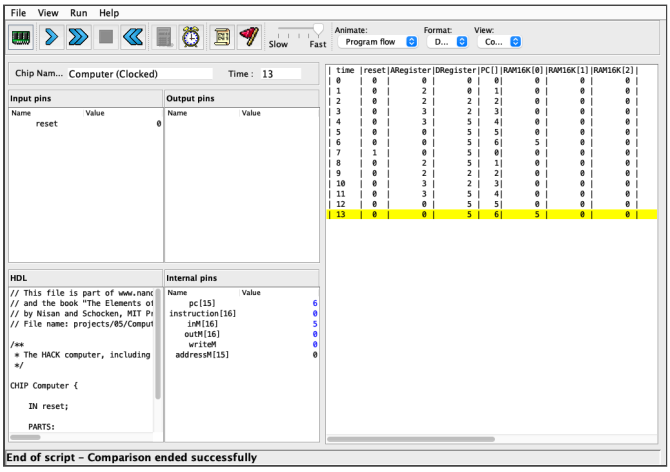
- **CPU.hdl:** Decodes 16-bit instructions into A-instructions and C-instructions, controls the A and D registers, sends inputs to the ALU, and updates the PC register based on the jump bits and ALU flags (`zr` and `ng`).
- **Memory.hdl:** Uses `DMux4Way`, `Or`, and `Mux4Way16` on `address[13..14]` to connect `RAM16K` (0 to 16383), `Screen` (16384 to 24575), and `Keyboard` (24576).
- **Computer.hdl:** Connects `ROM32K`, `CPU`, and `Memory` together. Note: For the large memory chips (`RAM16K`, `Screen`, `Keyboard`, and `ROM32K`), I used the built-in chips in the Hardware Simulator so the tests and screen output run smoothly.



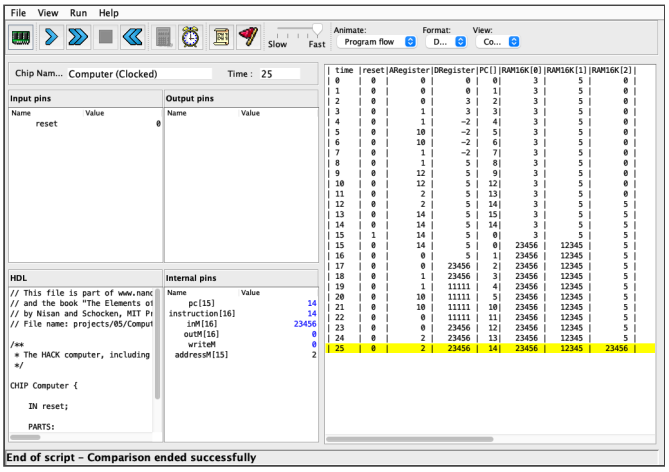
1. CPU.tst result



2. Memory.tst result

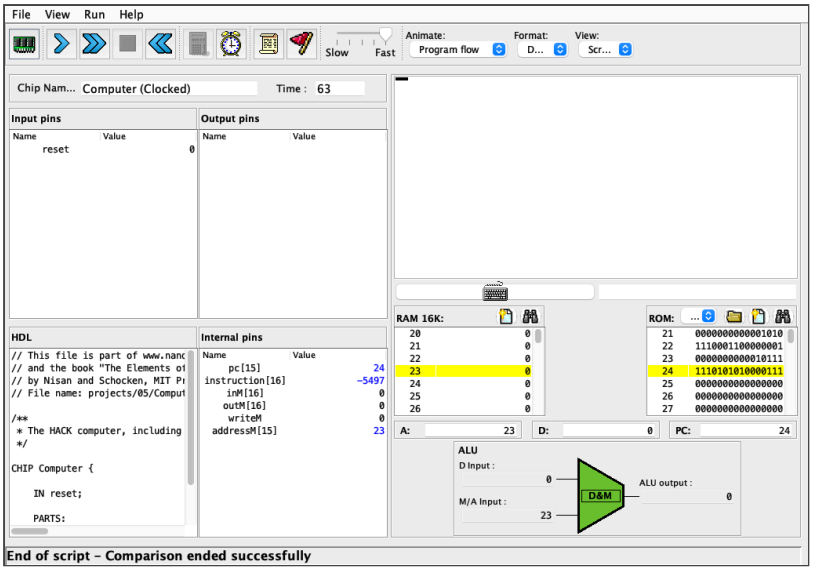


3. ComputerAdd.tst result



4. ComputerMax.tst result

Part A (Continued) - ComputerRect.tst and HDL Code



5. ComputerRect.tst result

CPU.hdl

```
CHIP CPU {
    IN  inM[16], instruction[16], reset;
    OUT outM[16], writeM, addressM[15], pc[15];
    PARTS:
    Not(in=instruction[15], out=isA);
    And(a=instruction[15], b=instruction[5], out=aluToA);
    Mux16(a=instruction, b=aluOut, sel=aluToA, out=inA);
    Or(a=isA, b=aluToA, out=loadA);
    ARegister(in=inA, load=loadA, out=outA, out[0..14]=addressM);
    And(a=instruction[15], b=instruction[12], out=selM);
    Mux16(a=outA, b=inM, sel=selM, out=outAM);
    And(a=instruction[15], b=instruction[4], out=loadD);
    DRegister(in=aluOut, load=loadD, out=outD);
    ALU(x=outD, y=outAM, zx=instruction[11], nx=instruction[10],
        zy=instruction[9], ny=instruction[8], f=instruction[7], no=instruction[6],
        out=aluOut, out=outM, zr=zr, ng=ng);
    And(a=instruction[15], b=instruction[3], out=writeM);
    Or(a=zr, b=ng, out=zrOrNg);
    Not(in=zrOrNg, out=pos);
    And(a=instruction[2], b=ng, out=jlt);
    And(a=instruction[1], b=zr, out=jeq);
    And(a=instruction[0], b=pos, out=jgt);
    Or(a=jlt, b=jeq, out=jle);
    Or(a=jle, b=jgt, out=jmpCond);
    And(a=instruction[15], b=jmpCond, out=loadPC);
    Not(in=loadPC, out=incPC);
    PC(in=outA, load=loadPC, inc=incPC, reset=reset, out[0..14]=pc);
}
```

Memory.hdl

```
CHIP Memory {
    IN in[16], load, address[15];
    OUT out[16];
    PARTS:
    DMux4Way(in=load, sel=address[13..14],
             a=loadRamA, b=loadRamB,
             c=loadScreen, d=loadKbd);
    Or(a=loadRamA, b=loadRamB, out=loadRam);
    RAM16K(in=in, load=loadRam,
           address=address[0..13], out=ramOut);
    Screen(in=in, load=loadScreen,
           address=address[0..12],
out=screenOut);
    Keyboard(out=kbdOut);
    Mux4Way16(a=ramOut, b=ramOut, c=screenOut,
              d=kbdOut, sel=address[13..14],
out=out);
}
```

Computer.hdl

```
CHIP Computer {
    IN reset;
    PARTS:
    ROM32K(address=pc, out=instruction);
    CPU(inM=inM, instruction=instruction,
        reset=reset, outM=outM,
        writeM=writeM, addressM=addressM,
        pc=pc);
    Memory(in=outM, load=writeM,
           address=addressM, out=inM);
}
```

Part B - Small Assembler in Python (assembler.py)

I wrote `assembler.py` in Python to translate `.asm` programs into 16-bit `.hack` files in two readings:

- **First reading:** Removes comments and blank lines, and records the ROM instruction address of each label like `(LOOP)` in the symbol table.
- **Second reading:** Converts A-instructions (`@value`) and C-instructions (`dest=comp;jump`) into 16-bit binary. Any new variable is assigned a RAM address starting from 16, 17, 18, and so on.
- **Extra feature for K = 65 (65 mod 3 = 2):** In the first reading, if the same label appears more than once, the assembler prints a warning message showing the line number.
- **Testing:** I tested `assembler.py` on `Add.asm`, `Max.asm`, `Rect.asm`, and `Pong.asm`. All generated `.hack` files matched the official assembler output completely.

assembler.py

```
def assemble(filename):
    with open(filename, "r") as f:
        raw_lines = f.readlines()

    symbol_table = dict(SYMBOLS)
    seen_labels = {}
    instructions = []
    rom_addr = 0

    # First reading: remove comments/blank lines and find labels
    # Extra feature for K = 65 (65 mod 3 = 2): print warning if same label is written twice
    for line_no, raw in enumerate(raw_lines, start=1):
        line = clean_line(raw)
        if not line:
            continue

        if line.startswith("(") and line.endswith(")"):
            label = line[1:-1]
            if label in seen_labels:
                print(f"Warning: label '{label}' written twice at line {line_no}")
            else:
                symbol_table[label] = rom_addr
                seen_labels[label] = rom_addr
        else:
            instructions.append((line_no, line))
            rom_addr += 1

    # Second reading: convert every instruction into 16 bits
    next_ram = 16
    output_lines = []

    for line_no, inst in instructions:
        if inst.startswith("@"):
            val = inst[1:]
            if val.isdigit():
                addr = int(val)
            else:
                if val not in symbol_table:
                    symbol_table[val] = next_ram
                    next_ram += 1
                addr = symbol_table[val]
            output_lines.append(format(addr, "016b"))
        else:
            dest, comp, jump = "", inst, ""
            if "=" in comp:
                dest, comp = comp.split("=", 1)
            if ";" in comp:
                comp, jump = comp.split(";", 1)
            binary = "111" + COMP_TABLE[comp] + DEST_TABLE[dest] + JUMP_TABLE[jump]
            output_lines.append(binary)
```

Part C - Write the Box Program (Box.asm)

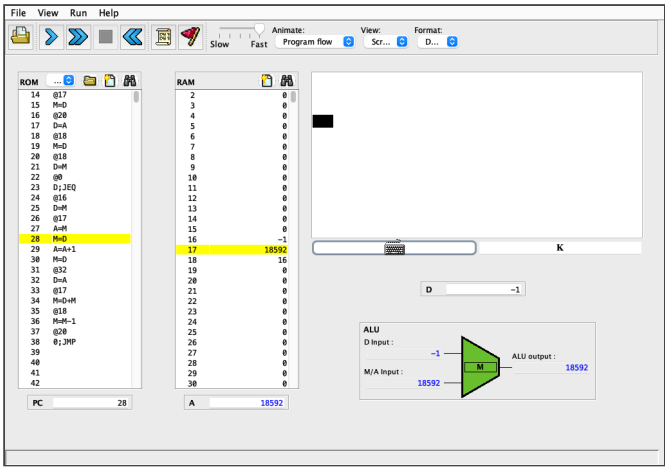
For my roll number `2501730365`, **K = 65**:

- **Start Row:** Row 65. Address = `SCREEN + 32 * 65 = 16384 + 2080 = 18464`.
- **Height:** 20 rows tall (rows 65 to 84).
- **Width:** `16 * ((65 mod 4) + 1) = 16 * (1 + 1) = 32` pixels wide (2 words per row from the left edge).

Box.asm

```
(LOOP)
    @KBD
    D=M
    @KEY_PRESSED
    D;JNE
(KEY_RELEASED)
    @color
    M=0
    @INIT_BOX
    0;JMP
(KEY_PRESSED)
    @color
    M=-1
(INIT_BOX)
    @SCREEN
    D=A
    @2080
    D=D+A
    @addr
    M=D
    @20
    D=A
    @rows
    M=D
(ROW_LOOP)
    @rows
    D=M
    @LOOP
    D;JEQ
    @color
    D=M
    @addr
    A=M
    M=D
    A=A+1
    M=D
    @32
    D=A
    @addr
    M=D+M
    @rows
    M=M-1
    @ROW_LOOP
    0;JMP
```

Box Screenshot in CPU Emulator



6. Box.hack running in CPU Emulator (Row 65, 20 rows tall, 32 pixels wide)